Multi-Product Game-Market Pricing: SAC and Feedback Control

Individual undergraduate capstone | Computer Science, National Cheng Kung University

Author: evanchang1217 (GitHub). Original capstone: 2nd place among 87 projects.

Research question and contributions

How can an NPC shop adjust multiple buy/sell prices while balancing trading targets, player-currency tracking, and price stability? This project builds a continuous-control system and evaluates Soft Actor-Critic (SAC) against validation-tuned feedback rules. Quantitative evidence comes from a heuristic market simulator.

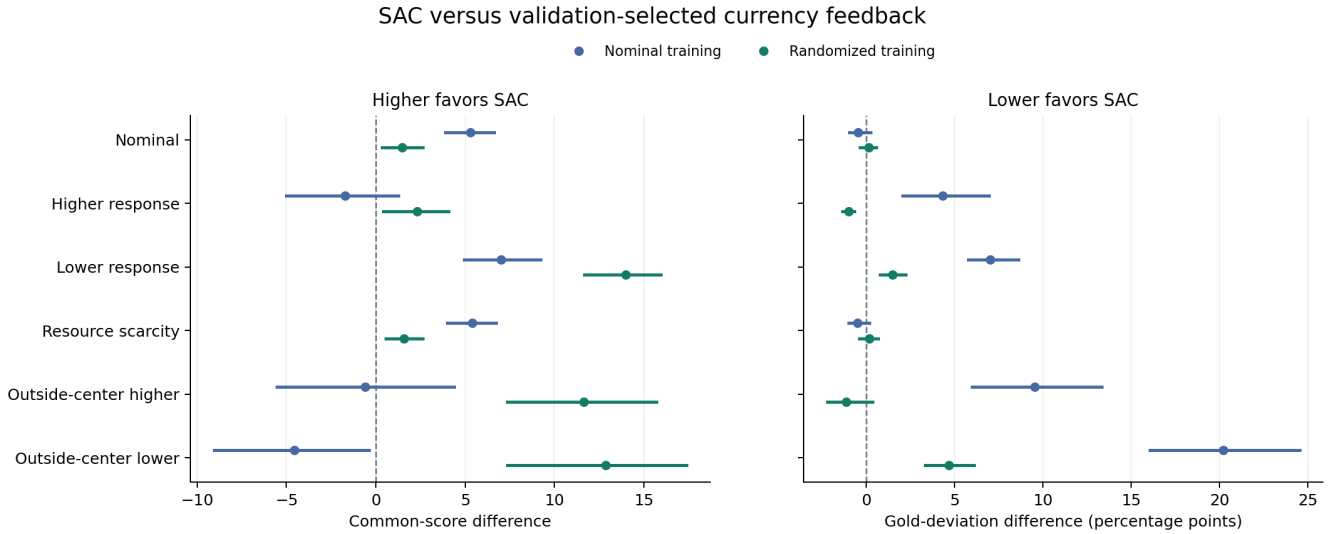
1. Build a reproducible pricing system. Model inventories, budgets, and stochastic trades; integrate SAC, a Django demonstration, and Minecraft status API reads. The maintained CLI and demo share one core and save configurations, policies, and logs.

2. Design and test the control objective. Audit the weak realized currency penalty, introduce explicit currency-setpoint tracking, and compare legacy, zero-currency, and revised objectives at equal training budgets.

3. Measure controller trade-offs. Compare five seeds per SAC condition with 36 feedback candidates across six scenarios, separating validation from testing and retaining unfavorable outcomes and paired uncertainty estimates.

Objective ablation: At equal 10k budgets, revised SAC lowers currency error 88.9% versus legacy SAC, retaining 96.3% of units. Feedback still scores better.

The award concerns the original individual capstone. Follow-up work is separately dated and used AI assistance for portions of coding and evaluation. The contribution uses the existing SAC algorithm. Authorship and history.



Five models per SAC condition; 30k steps; 40 paired initial markets per scenario.
Saved exploratory 95% crossed-bootstrap intervals (10,000 resamples); no multiplicity correction.
One controller selected on the validation mixture is used in every test scenario; higher score does not imply lower currency error.

Code and results: github.com/evanchang1217/Soft-actor-critic-dynamic-pricing

Main comparison: five seeds, fixed 30k budget

Each SAC model completes 30,000 interactions. Twenty validation markets under three response conditions select feedback parameters before testing on 40 new initial markets under six scenarios. Episodes use 200 players, three goods, 100 steps, and a fixed 5% fee. The table below reports nominal test markets.

Method	Common score ↑	Currency error ↓	Units traded	Sell-price change ↓
Feedback (validation-tuned)	−36.01	3.39%	183,370	2.15%
SAC (nominal training)	−30.70	2.91%	227,090	1.40%
SAC (response-randomized)	−34.54	3.52%	257,145	1.18%

Supported advantage. Nominal SAC improves the common score over the score-selected feedback rule by +5.30, exploratory 95% interval [3.88, 6.62]. Its lower mean currency error does not establish an advantage: that difference interval includes zero. Differences use unrounded values; displayed means need not subtract exactly.

What the score means. All methods receive the same negative sum of trading-target, aggregate shop-price-index-change, and currency-deviation penalties over 100 steps. Less negative is better. Trading penalties track specified targets; a higher score need not mean more trading or greater welfare.

Robustness and the cost of learned pricing

- Training distribution: response randomization reduces higher-response SAC currency deviation from 9.06% to 3.74%, versus feedback's 4.75%, while reducing nominal score. Outside-center lower response remains a failure: 6.69% deviation versus feedback's 2.02%.

- Stronger activity comparator: in a validation-fixed, approximately error-matched nominal pair, randomized SAC and volume-oriented feedback have 3.52% / 3.28% error, 257,145 / 324,326 units, and 1.18% / 2.14% sell-price changes. SAC is smoother and trades less; the chosen score advantage does not imply dominance over feedback.

Scope and reproducibility

Currency error averages abs(total_gold / initial_total_gold - 1). Sell-price change averages absolute relative changes across goods and steps, excluding shop buy prices. Policies execute separately and are averaged for reporting, not as an action ensemble.

The main-study simulator lacks real-player calibration and a consumption/utility model. A later behavior audit adds illustrative consumption/satiation, under which the equal-budget SAC score advantage reverses. Original-market 1,000-step tests retain score gains but expose currency/distribution trade-offs. Currency is a chosen setpoint; these simulations do not establish welfare or real-game inflation control. Limited game testing is author-reported; public API code reads status. Five seeds and unadjusted intervals support exploratory conclusions.

Configurations, models, data, source snapshots, and verification code are retained. Main-study verification reconstructs statistics and replays 66 selected episodes; full replay is separate. Results · Methods · Reproduction · Poster supplement.